Security Assessment

Final Report

Kvault – PR78 – Vault Rewards

March 2026

Prepared for Kamino

Table of Contents

Project Summary 3
Project Scope 3
Project Overview 3
Protocol Overview 3
Assessment Methodology 4
Findings Summary 5
Severity Matrix 5
Detailed Findings 6
Medium Severity Issues 7
M-01: Stale `last_issuance_ts` Causes Instant Reward Dump on Re-topup 7
Informational Severity Issues 9
I-01: Duplicate call of `refresh_rewards` 9
Disclaimer 10
About Certora 10

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Kvault - PR78 - Vault Rewards	https://github.com/Kamino-Finance/kvault-private	dabe9a26f8e2c1b77e596f292be62b0a3a1cfd94	Solana

Project Overview

This document describes the security review of **Kvault - PR78 - Vault Rewards**. The work was undertaken from **March 10th, 2026** to **March 13th, 2026**.

The following contract list is included in our scope:

Changes made in src/* in the the following PR

The team performed a manual audit of all the files in scope**.** During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Rewards for the same token as the vault, accumulated and accounted for within the vault, increasing the value of the vault share directly, giving the users a more convenient reward accumulation for tax reporting purposes. (no extra transactions where rewards are claimed separately as with the Farms program)

- create a RewardInfo { reward_vault, reward_per_second } field in a Vault
- on every interaction do -> refresh_rewards()
- which does tokens_available += rps * seconds passed
- needs handlers: handler_init_reward_vault, handler_update_vault { rps }, handler_topup_reward_vault

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	1	1	1
Low	-	-	-
Informational	1	1	1
Total	2	2	2

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
M-01	Stale “last_issuance_ts” Causes Instant Reward Dump on Re-topup	Medium	Fixed
I-01	Duplicate call of `refresh_rewards`	Informational	Fixed

Medium Severity Issues

M-01: Stale `last_issuance_ts` Causes Instant Reward Dump on Re-topup

Severity: Medium	Impact: High	Likelihood: Low
Files: programs/kvault/src/operations/vault_operations.rs:833	Status: Fixed	

Description:

When rewards_available is fully exhausted, last_issuance_ts is not reset. It freezes at the timestamp of the last distribution. On re-topup, the accumulated "dry period" seconds cause all newly added rewards to be distributed in a single transaction rather than gradually, breaking the intended drip schedule and allowing for a stepwise jump farming.

Attack Scenario

1. Vault has rps = R, rewards exhaust at time T. last_issuance_ts freezes at T.
1. Admin tops up new rewards at T + Δ (where Δ is large). last_issuance_ts stays at T.
1. Attacker deposits large amount right before `topup_rewards`.

1. A keeper calls topup_rewards with new rewards. refresh_rewards fires with seconds_passed = Δ, dumping the entire new topup into token_available at once.

1. Attacker immediately withdraws, capturing the majority of the rewards that were meant to distribute over the next `new_topup / rps` seconds.

Recommendations:

In refresh_rewards, when rewards_available reaches 0 after distribution, reset last_issuance_ts = 0. The existing logic in topup_rewards then correctly sets it to current_ts on the next topup, restarting the drip from that moment.

Customer Response:

Fixed in PR136.

Fix Review:

Fix confirmed.

Informational Severity Issues

I-01: Duplicate call of `refresh_rewards`

Files:

programs/kvault/src/operations/vault_operations.rs::480

Description:

refresh_rewards is called twice in some flows, which may be optimized to save CU.

Flows:

• update_vault_config → The handler calls refresh_rewards, which is also called inside the internal update_vault_config → VaultConfigField::RewardPerSecond field

• invest → The handler calls refresh_rewards, which is also later called in the internal func

Customer Response:

Fixed in 7bedc8.

Fix Review:

Fix confirmed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.